

Helix OTA

Field	Value
Revision	1
Created	2026-06-07
Last modified	2026-06-07
Status	active
Status summary	Project root README for Helix OTA â€” a universal, decoupled over-the-air update system (Go control plane + per-OS client SDKs/agents). First target is Android 15 on Orange Pi 5 Max. The project is currently in its specification/research phase: the authoritative design lives in the spec corpus under <code>docs/research/main_specs/</code>, and implementation follows that corpus. The spec corpus entry point at docs/research/main_specs/README.md is referenced as the corpus index but is not yet present (UNVERIFIED); until it lands, start from the master design (Â§ links below). The six <code>NEW o t a - *</code> submodule repositories are not yet created. HelixConstitution clause numbers are carried from the corpus convention and are UNVERIFIED against the authoritative constitution text.
Issues	
Fixed	N/A (initial revision).
Continuation	Add the corpus index README .md; complete the per-component 1.0.0-MVP specs and the open ADRs; create the six PUBLIC <code>o t a - *</code> repos on GitHub + GitLab; then begin implementation under the corpus.

Table of contents

1. [What is Helix OTA](#)
2. [Status](#)
3. [Mandated technology stack](#)
4. [Locked architecture](#)
5. [Specification corpus](#)
6. [Reusable submodules](#)
7. [Governance â€” HelixConstitution](#)
8. [Documentation export pipeline](#)
9. [Repository layout](#)
10. [Tracked-Items + Status Documents](#)
11. [License](#)

Per HelixConstitution Â§11.4.61 (UNVERIFIED clause number), every document carries the metadata table above followed by this table of contents.

1. What is Helix OTA

Helix OTA is a **universal, generic, deeply decoupled** over-the-air (OTA) update system. It comprises a server **control plane**, per-OS **client SDKs/agents**, and a dashboard, designed to be embeddable into any operating system.

- **First target:** Android 15 (all flavors/variants) on the **Orange Pi 5 Max**, where the build pipeline emits flashing images plus an OTA update `.zip` and mandatory hash files.
- **Future targets:** Linux (all flavors), Windows, and other upstream OSes via pluggable OS adapters. These are research/standards-survey only at this stage (`1.X-linux/`, `1.X-windows/`, `1.X-other-os/`).

Operator-stated hard guarantees (from the master design):

- **Zero system corruption** â€” an update must never brick a working device.
- **Safe upload** â€” every OTA artifact passes mandatory validation before it can be deployed.
- **Granular rollout** â€” deploy all at once, or in steps (5%, 10%, 30%, â€¦ 100%).
- **Observability** â€” tracking/measurement/critical-data capture to detect and report problems.
- **Scalability** â€” single board to millions of devices.

2. Status

Specification / research phase. This repository currently contains the design and research corpus, the documentation export pipeline, and the submodule scaffolding tooling â€” **not** the implementation. Architecture and stack are locked (see below); component-level decisions that require evidence are deliberately deferred to the open ADRs. Implementation follows the corpus.

No claim of working server, agent, or `ota-*` submodule code is made here; those do not yet exist (anti-bluff, HelixConstitution Â§7.1 / Â§11.4.6, UNVERIFIED clause numbers).

3. Mandated technology stack

The stack is locked by operator decision (master design Â§2 D6, Â§3):

- **Language / runtime:** Go (control plane, rollout engine, validators); Kotlin/KMP (Android agent).
- **HTTP framework:** **Gin** (`gin-gonic`).
- **Transport:** **HTTP/3 (QUIC)** primary with automatic fallback to **HTTP/2** + standard compression; **Brotli** content compression with negotiated **gzip** fallback for older clients.
- **API surface:** **REST** is the mandated primary surface (`/api/v1`) plus mandatory compatibility interfaces; gRPC is optional/internal only.
- **Persistence:** PostgreSQL (relational), MinIO/S3 (artifact blobs), optional Redis caching only where a measured need exists.
- **Observability:** OpenTelemetry; Prometheus/Grafana surface.

4. Locked architecture

Locked by operator decision (master design Â§2 D2): **device-side native Android A/B** (`update_engine` + AVB/dm-verity + automatic boot-failure rollback) **plus a custom**,

decoupled Go control plane; an OSS engine is wrapped only where it adds value (and that choice is deferred to an evidence-based ADR, D3).

Three planes plus infrastructure, with two deliberately extractable seams – the **OS-adapter** seam (universality) and the **rollout-engine** seam (OS-agnostic campaigns):

- **Control plane** (Go + Gin, HTTP/3 + HTTP/2, Brotli): REST API, artifact intake + validation, rollout engine, device registry, telemetry ingest, auth (OAuth2/JWT, RBAC).
- **Data plane**: PostgreSQL, MinIO/S3, OpenTelemetry/Prometheus.
- **Device** (Android 15 on Orange Pi 5 Max): KMP OTA agent (poll/download/verify), AOSP update_engine (Virtual A/B), AVB/dm-verity + boot_control.
- **Dashboard**: React (login, upload, rollout, fleet health).

Each unit follows the decoupling principle (one purpose, well-defined interface, independently testable; §11.4.28, UNVERIFIED).

5. Specification corpus

The authoritative design and research live under [docs/research/main_specs/](#). The intended corpus index is [docs/research/main_specs/README.md](#) (not yet present – UNVERIFIED). Until it lands, the canonical entry points are:

- **Master design**: [docs/research/main_specs/00-master/2026-06-07-helix-ota-design.md](#)
- **Submodule reuse map**: [docs/research/main_specs/00-master/submodule_reuse_map.md](#)
- **Documentation standards**: [docs/research/main_specs/00-master/documentation_standards.md](#)
- **Architecture Decision Records (open)**: [docs/research/main_specs/research/adr/](#)

Corpus phases: 00-master/, research/, additions/, 1.0.0-mvp/, 1.0.1-staged-rollout/, 1.X-linux/, 1.X-windows/, 1.X-other-os/.

6. Reusable submodules

Helix OTA is **catalogue-first** (HelixConstitution §11.4.74, UNVERIFIED): every component is satisfied by an existing verified catalogue submodule where one exists; a NEW submodule is created only where the catalogue has no cover. See the [submodule reuse map](#) for the full component + submodule bindings.

Six NEW reusable, independently-versioned submodules are introduced because no catalogue submodule covers their purpose. Per locked decision D4 they get **PUBLIC** repositories on both GitHub and GitLab. **These repos are not yet created** (UNVERIFIED – URLs below are the planned canonical locations):

Submodule	Purpose (decoupled boundary)	GitHub	GitLab
ota-protocol	Shared wire types, manifest schema, status/event enums (Go + KMP); pure	https://github.com/HelixDevelopment/ota-protocol	https://gitlab.com/helixdevelopment1/ota-protocol

Submodule	Purpose (decoupled boundary)	GitHub	GitLab
	contracts, no business logic.		
ota-artifact-validator	Structure/hash/signature/metadata validation pipeline; OS-aware via plugins, no transport.	https://github.com/HelixDevelopment/ota-artifact-validator	https://gitlab.com/helixdevelopment1/ota-artifact-validator
ota-rollout-engine	Staged-rollout + halt/advance logic, OS-agnostic; no HTTP, pure engine + storage port.	https://github.com/HelixDevelopment/ota-rollout-engine	https://gitlab.com/helixdevelopment1/ota-rollout-engine
ota-update-engine-bridge	Wrapper over AOSP update_engine / boot_control; Android-only, thin, testable.	https://github.com/HelixDevelopment/ota-update-engine-bridge	https://gitlab.com/helixdevelopment1/ota-update-engine-bridge
ota-android-agent	KMP device agent (poll/download/verify/apply/report); consumes protocol + bridge.	https://github.com/HelixDevelopment/ota-android-agent	https://gitlab.com/helixdevelopment1/ota-android-agent
ota-telemetry-schema	Telemetry event/metric schema + codecs; shared by server + agents, no transport/storage.	https://github.com/HelixDevelopment/ota-telemetry-schema	https://gitlab.com/helixdevelopment1/ota-telemetry-schema

Reused catalogue submodules (verified canonical names from documentation standards §9) include auth, security, database, Storage, observability, eventbus, ratelimiter, middleware, http3, mdns, recovery, Herald, config, discovery, cache, docs_chain / Document / Formatters, containers, and the KMP set Auth-KMP, Security-KMP, Storage-KMP, Config-KMP. New repos can be bootstrapped with [scripts/scaffold_submodule.sh](#), which scaffolds the README (metadata table), Apache-2.0 license, .gitignore, docs/ and tests/ placeholders, then configures and pushes the GitHub + GitLab remotes.

7. Governance “ HelixConstitution

Helix OTA is governed by the **HelixConstitution**, the standards the corpus is authored against. Key rules applied throughout (clause numbers carried from the corpus convention and **UNVERIFIED** against the authoritative constitution text):

- **Four-layer testing + mutation meta-test** (§1) “ source-presence gate ‘ artifact gate ‘ runtime/integration ‘ mutation meta-test, with no-bluff positive evidence.
- **Anti-bluff** (§7.1 / §11.4.6) “ no fabricated facts; anything unconfirmed is marked UNVERIFIED. This README follows that rule.
- **Catalogue-first reuse** (§11.4.74) and the **decoupling principle** (§11.4.28).
- **Containers substrate** (§11.4.76).
- **Metadata table + table of contents on every document** (§11.4.61).

- **Subagent-driven authoring with a mandatory code-review gate** (Â§11.4.20 / Â§11.4.125).
- **Multi-upstream mirroring** (Â§2 / Â§2.1) â€” every repo is pushed to GitHub and GitLab.

8. Documentation export pipeline

Canonical source is **Markdown + Mermaid**. The multi-format export pipeline [scripts/export_docs.sh](#) converts the corpus into HTML and DOCX (plus PDF when a LaTeX/PDF engine is available) and renders embedded Mermaid diagrams to SVG and PNG, writing outputs into per-directory `_exports/` folders.

It **degrades gracefully and honestly**: each renderer (pandoc, mermaid-cli, a LaTeX/PDF engine) is probed at runtime, missing tools are logged as SKIPPED rather than failing the run, and a format is only reported as produced if its file actually exists (export pipeline notes, Â§11.4.65 / Â§7.1, UNVERIFIED clause numbers).

```
# Export the whole corpus (default target) or a single file/dir:
scripts/export_docs.sh
scripts/export_docs.sh docs/research/main_specs/00-master/2026-06-07-helix
```

9. Repository layout

```
.
â€œâ€œâ€œâ€œâ€œ README.md           this file
â€œâ€œâ€œâ€œâ€œâ€œ LICENSE          Apache-2.0
â€œâ€œâ€œâ€œâ€œâ€œ docs/
â€œ,   â€œâ€œâ€œâ€œâ€œâ€œ request/      operator request inputs
â€œ,   â€œâ€œâ€œâ€œâ€œâ€œâ€œ research/main_specs/ authoritative design + research corp
â€œâ€œâ€œâ€œâ€œâ€œâ€œ scripts/
â€œ,   â€œâ€œâ€œâ€œâ€œâ€œâ€œ export_docs.sh multi-format documentation export pi
â€œ,   â€œâ€œâ€œâ€œâ€œâ€œâ€œ scaffold_submodule.sh scaffold + push a new reusable submo
â€œâ€œâ€œâ€œâ€œâ€œâ€œâ€œâ€œâ€œâ€œ upstreams/ multi-upstream remote host definitions
```

10. Tracked-Items + Status Documents

Doc-map for the canonical tracker documents plus the key spec/status documents in the active spec corpus (`docs/research/main_specs/`). Each row links the canonical Markdown source plus its HTML/PDF exports where those siblings exist on disk; `Revision` and `Last modified` are read from each documentâ€™s own metadata header (â€œ = absent). Every linked path was verified to exist on disk (HelixConstitution Â§11.4.57 / Â§7.1 anti-bluff, UNVERIFIED clause numbers).

Document	Last modified	Revision	Markdown	HTML	PDF
Issues â€œ					
open workable items	2026-06-10T18:30:00Z	2	md	html	pdf
Issues Summary â€œ					
open, short-form	2026-06-10T18:30:00Z	2	md	html	pdf
	2026-06-10T18:30:00Z	2	md	html	pdf

Document	Last modified	Revision	Markdown	HTML	PDF
Fixed “closed workable items”					
Fixed Summary “closed, short-form”	2026-06-10T18:30:00Z	2	md	html	pdf
Session Resumption “canonical entry point”	2026-06-10T12:00:00Z	2	md	html	pdf
Spec-corpus continuation / handoff	2026-06-10	3	md	html	pdf
Additions synthesis (gap closure)	2026-06-08	3	md	html	pdf
1.0.0-MVP API “operational endpoints”	2026-06-08	1	md	html	pdf
1.0.0-MVP API “implemented endpoints”	2026-06-08T00:00:00Z	1	md	html	pdf
1.0.0-MVP dashboard design	2026-06-08	1	md	html	pdf
1.0.1 staged rollout “overview (README)”	2026-06-08	2	md	html	pdf
1.0.1 staged rollout “rollout engine”	2026-06-08	1	md	html	pdf
1.0.1 staged rollout “migration 002 design”	2026-06-08	1	md	html	pdf
1.0.1 staged rollout “device TUF”	2026-06-08	1	md	html	pdf
1.0.1 staged rollout “rollback UX”	2026-06-08	1	md	html	pdf
1.0.3 delta updates “overview (README)”	2026-06-08	1	md	html	pdf

Document	Last modified	Revision	Markdown	HTML	PDF
1.0.3 delta updates â€™ design	2026-06-08	1	md	html	pdf
Repo public-visibility audit (gap G11)	2026-06-08T00:00:00Z 1		md	html	pdf

11. License

Apache-2.0. See [LICENSE](#).